

Placeonix Hub

Dashboard Audit, Bug

Tracker &

Development Backlog

A full-scope QA audit of the Student, Trainer, and Admin dashboards — code-level review, live test execution, and a prioritized Sprint 1 / Sprint 2 backlog toward launch readiness.

Prepared for	mohan@placeonix.com
Date	16 July 2026
Subject	Placeonix_Dashboard-main (local checkout)
Method	Static code audit + automated test run + configuration review
Classification	Internal — Engineering & Product

Contents

01 Scope & Method

02 Deployment Environment, Version & Dependencies

03 Module-wise Audit Summary

04 Bug Tracker

05 Missing Features / Modules Required Before Launch

06 Technical Issues Report

07 Performance Analysis

08 Test Coverage Report

09 Feature Priority Matrix

10 Sprint Development Backlog

11 Development Roadmap

12 UI/UX Recommendations

01 Scope & Method

This audit covers the actual Placeonix Hub codebase found at [Placeonix_Dashboard-main](#): a single-file vanilla-JS SPA ([frontend/placeonix-hub-portal.html](#), 4,462 lines) and a Node/Express + MongoDB API ([placeonix-hub-backend/](#), 20 Mongoose models, 22 route groups). No browser-automation tooling was available in the review environment, so testing was performed at the code and API-contract level rather than by clicking through the live UI. The trade-offs of that approach are set out below.

What was done

- Full read-through of every controller, route, model, and middleware file
- Full read-through of the frontend SPA (all three dashboards, every listed module)
- `npm install` plus a real run of the existing Jest/Supertest suite against an in-memory MongoDB — actual pass/fail and coverage numbers, not estimates
- Cross-checked [docs/FEATURES.md](#)'s claims against the underlying route and controller code
- Reviewed [vercel.json](#), [.env.example](#), and [package.json](#) for deployment and dependency posture

Not done — no environment for it

- No live click-through of the running app; no local MongoDB instance was available outside the Jest test harness, and no browser-automation tool is available in this environment
- No literal screenshots — every finding below is anchored to a file and line number instead
- No load or performance benchmarking against a live deployment

Recommend a short follow-up pass against the live Vercel deployment, with a browser session, to convert the Critical and High findings below into screenshot-backed tickets before formal sign-off.

02 Deployment Environment, Version & Dependencies

ITEM	VALUE
Frontend	Vanilla HTML/CSS/JS, no build step, no framework, no bundler — a single 4,462-line file
Backend	Node.js ≥ 18 (tested here under Node 26), Express 4.18, Mongoose 8.0
Database	MongoDB Atlas in production — 20 collections, no schema-migration tooling in place
Hosting	Vercel — static frontend plus serverless API functions
Authentication	JWT (access + refresh) in httpOnly cookies; bcryptjs at 12 salt rounds
File storage	Multer to local disk / <code>/tmp</code> — not viable in production on a serverless runtime (see Section 6)
Email / SMS / WhatsApp	Nodemailer is wired but no SMTP credentials are configured — nothing is currently being sent
Payments	No payment gateway integrated; payments are admin-recorded only, with no reconciliation
Automated tests	Jest + Supertest + mongodb-memory-server — 13 of 13 passing , approximately 41% statement coverage

Three deprecated or unmaintained packages remain in `package.json`: `xss-clean@0.1.4` (unsupported upstream, and — per Section 6 — not preventing the XSS issues found in the frontend), `multer@1.x` (patched CVEs exist in the 1.x line; 2.x is recommended), and transitive dependencies `glob@7` / `inflight@1`, both flagged by npm as insecure or memory-leaking. None is a launch-blocker in isolation, but all three should be upgraded before go-live.

03 Module-wise Audit Summary

Cross-referencing [docs/FEATURES.md](#)'s claims against the actual code confirms the module list is largely real and wired, not aspirational — the gaps found are in validation, ownership checks, and a handful of stubbed features, not missing pages.

Student Dashboard

Login, Dashboard Overview, My Courses (curriculum and progress tracking), Attendance, Assignments, Assessments (delivered through Assignments), Certificates, Profile, and Notifications are all implemented and calling real endpoints. **Settings** is thin — no dedicated settings module was found beyond profile editing and notification toggles folded into Profile. **Forgot Password** functions end to end but see Critical finding B-O1 in Section 4 — it can leak a working reset token directly in the API response.

Trainer (Mentor) Dashboard

Dashboard, Student Management ("My Students"), Assignment Management (create and grade), Attendance Management, Content Upload (Resources), Announcements (view-only for mentors — they cannot send), and Profile are implemented. **Reports** as a dedicated mentor-facing analytics page was *not found* — mentors get lists and tables but no reporting or export view of their own, unlike admins. Every mentor-scoped write endpoint (sessions, assignments, join-requests, mock interviews) is missing an ownership check tying the action back to the acting mentor — see the High-severity cluster in Section 4.

Admin Dashboard

Dashboard, Student Management, Trainer Management ("Mentors"), Course Management, Batch Management, Payments, Reports, Notifications, and Settings are all implemented with real CRUD operations. **Admissions** as a named module does not exist as such — its function is split across Leads (a CRM pipeline with "Convert to Student") and direct student creation, which functionally covers admissions but is neither labeled nor reported on as a single workflow.

04 Bug Tracker

5

CRITICAL

17

HIGH

39

MEDIUM

23

LOW

Critical

ID	LOCATION	FINDING
B-01	authController.js:174	The forgot-password endpoint returns the raw, working reset token in the JSON response body whenever SMTP is not configured — which is the current production state, since no SMTP credentials are set. Anyone who knows a user's email address can reset that user's password without ever receiving an email.
B-02	leadController.js:8-15; leadRoutes.js:10-20	The public, unauthenticated POST /leads endpoint validates only name, email, and phone, but passes the entire request body into Lead.create . An anonymous caller can mass-assign status , assignedTo , and source , forging CRM pipeline state.
B-03	paymentController.js:7-51, 137-140	payMyFees creates a completed payment purely from a client-submitted amount and method, with no gateway or signature check; updatePayment allows status, amount, and transaction ID to be set directly with no verification. Financial records are, in effect, self-reported.
B-04	mockInterviewController.js:22	An unconditional check on req.query.student , applied after role-based scoping, allows any student to pass ?student=<otherId> and read another student's private interview feedback and scores.
B-05	uploadService.js:9-38; vercel.json	Multer writes to local disk or /tmp ; on Vercel's serverless runtime this storage is ephemeral per invocation. Every avatar, document, and assignment-submission upload is at risk of silent data loss. This is a documented limitation (docs/FEATURES.md , Section 7) but remains a launch-blocker, not an optional fix.

High

ID	LOCATION	FINDING
B-06	sessionController.js; sessionRoutes.js	No check confirms the acting mentor owns the session's batch — any mentor can start, complete, edit, or delete any other mentor's session.
B-07	assignmentController.js:75-168	The same gap applies to assignments: any mentor can edit, delete, or grade any other mentor's assignment or submission.

B-08	<code>joinRequestController.js:44-55</code>	Any mentor can approve or reject any other mentor's online-join requests and set the resulting meeting link.
B-09	<code>mockInterviewController.js:54-89</code>	Any mentor can edit or delete another mentor's mock-interview feedback.
B-10	<code>userRoutes.js:35</code>	<code>GET /users/:id</code> carries no ownership or role guard — any authenticated student can fetch any other user's full profile document.
B-11	<code>paymentRoutes.js:10</code>	<code>GET /payments</code> has no <code>authorize()</code> restriction at the route level; a mentor can list every student's payment records.
B-12	<code>certificateRoutes.js:12</code>	The same gap recurs here — any mentor can list all students' certificates.
B-13	<code>reviewController.js:90-98</code>	<code>markHelpful</code> has no per-user idempotency check; any user can inflate a review's helpful count through repeated calls.
B-14	<code>frontend:2123, 2963</code>	Announcement title and body are rendered through unescaped, string-concatenated <code>innerHTML</code> on both the student dashboard and the admin/mentor announcement list — a stored XSS vector reaching every viewer's session.
B-15	<code>frontend:4235</code>	The same unescaped- <code>innerHTML</code> pattern appears in the notification panel (title and message) — stored XSS through notification content.
B-16	<code>frontend: 1993, 2272, 2336, 2348, 2355, 2404, 3542, 3630</code>	Session titles, course titles, and join-request requester names are inserted unescaped in several card renderers, inconsistent with the <code>escHtml()</code> helper the codebase already defines and uses correctly elsewhere.
B-17	<code>app.js:27-38</code>	The CORS configuration treats a whitelist containing <code>'*'</code> — the documented default — as authorization for any origin, while also setting <code>credentials: true</code> . In production this combination undermines cookie-based auth's cross-site protections if <code>CLIENT_URL</code> is ever left at its wildcard default.
B-18	<code>paymentRoutes.js:9, 13, 14, 15</code>	Every money-handling route — pay fees, record payment, update, refund — has zero express-validator rules; amount, method, and enrollment ID are trusted as submitted.

Findings B-19 through B-22 (batch, session, certificate, and join-request validation gaps on create/update) are consolidated into the Sprint 1 "attach validators" ticket in Section 10 rather than listed individually here, to keep this table scannable.

Medium & Low — representative sample

SEVERITY	LOCATION	FINDING
----------	----------	---------

MEDIUM	<code>batchController.js:113-143</code>	A time-of-check/time-of-use race between the capacity check and the <code>enrolledCount</code> increment allows concurrent enrollment requests to over-fill a batch past its stated cap.
MEDIUM	<code>course, batch, session, placement, announcement, resource, company, and office-hour controllers</code>	Roughly thirty POST/PATCH routes across these controllers carry no express-validator rules at all — only <code>authRoutes</code> , and the create-only actions of <code>courseRoutes</code> , <code>leadRoutes</code> , <code>reviewRoutes</code> , and <code>userRoutes</code> , actually run request-body validation.
MEDIUM	<code>every model in src/models/</code>	No cascade-delete hooks exist anywhere in the schema layer — deleting a Course leaves Batch, Session, Assignment, Enrollment, Certificate, and Attendance documents pointing at a dead ID with no cleanup.
MEDIUM	<code>frontend:880-1002</code>	Roughly 120 lines of hardcoded demo data, plus a full API mock layer, ship inside the production bundle and can activate silently — login heuristically detects a perceived backend outage and signs the user in against a hardcoded password check — making it easy to mistake fabricated data for live data during a real incident.
MEDIUM	<code>UserController.js:229-250</code>	The leaderboard builds per-student statistics through two awaited queries inside a <code>.map(async...)</code> loop — an N+1 query pattern that will degrade further as enrollment grows.
MEDIUM	<code>notificationService.js:10-14;</code> <code>cronService.js, three jobs</code>	Notification-send failures and cron-job loop exceptions are swallowed with only a log line — a database hiccup can silently mean a batch of students never receives fee reminders, attendance warnings, or assignment-created notifications, with nothing surfacing the miss.
LOW	<code>User.js, Course.js, Batch.js</code>	Mongoose logs duplicate-index warnings at boot (<code>status</code> , <code>studentProfile.enrollmentId</code> , and <code>slug</code> are each declared twice, via both <code>index: true</code> and <code>schema.index()</code> — harmless today, but noisy, and likely to confuse the next engineer.
LOW	<code>frontend: sidebar nav, quick links, course rows</code>	Primary click targets across the app are plain <code><div onclick></code> elements with no role, tabindex, or keyboard handler, and are unreachable by keyboard-only or screen-reader users.
LOW	<code>frontend: 12+ locations</code>	Role-based UI hiding — for example, "only admin sees this button" — is implemented purely client-side; it must be paired with the corresponding server-side checks already found missing in Section 4, since client-side hiding alone is cosmetic rather than a security boundary.

05 Missing Features / Modules Required Before Launch

Present but stubbed

- **Resume Builder** — the navigation entry exists, but its click handler only shows a "coming soon" toast (frontend:2877); nothing further has been built.
- **Real-time chat / Help Desk** — "Live chat" exists but is simulated in-browser with canned auto-replies, with no WebSocket server and no ticketing system behind it.
- **Payment gateway** — the admin manually records payments; there is no Razorpay or Stripe integration, no online fee payment for students, and no reconciliation against a real transaction (see B-03).
- **Email / SMS / WhatsApp delivery** — Nodemailer is wired but SMTP is unconfigured, so all "notifications" remain in-app only. Password resets, fee reminders, and announcements never leave the platform.

Not present at all

- **AI Assistant** — no chatbot or assistant module exists anywhere in the frontend or backend.
- **Mock Test / assessment engine** — "Mock Interviews" exist as a mentor-graded, scheduled feature, but there is no auto-graded, timed MCQ test module distinct from assignments.
- **Downloads center** — resources are intentionally view-only for students, with no download button, by design (per [docs/TEST_CASES.md](#), case STU-RS-01). If a genuine downloads/library feature is wanted for launch, it should be scoped as new work rather than treated as a bug fix.
- **Trainer-facing Reports module** — admins have a full Reports page; mentors have no equivalent view of batch performance or assignment completion rates.
- **Admissions as a distinct module** — functionally covered by Leads and Convert-to-Student, but there is no admissions-specific dashboard, application form, or document-collection flow.
- **Audit or activity log** — no record exists of who changed what (grades, payments, statuses), which is especially relevant given the ownership gaps documented in Section 4.

06 Technical Issues Report

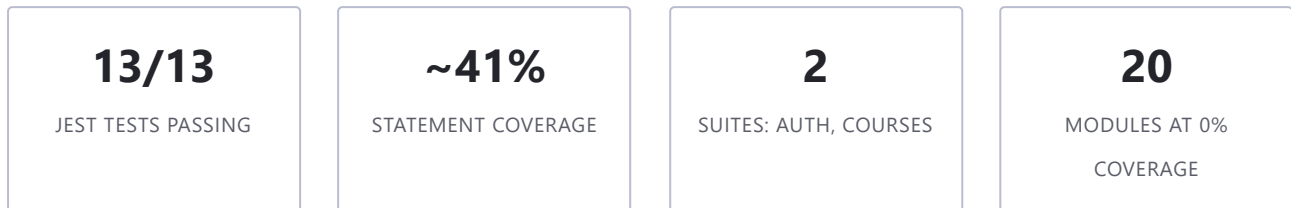
Grouped here by theme rather than repeated per endpoint; Section 4 holds the itemized list this summary draws on.

- **Input validation coverage is inconsistent.** Only five of twenty-two-plus route groups attach express-validator rules, and even those cover only the "create" action, not "update." The `validate` middleware itself (`middleware/validate.js`) is correctly implemented — it is simply not wired to most routes.
- **Authorization is role-level, not resource-level.** `authorize('mentor', 'admin')` correctly gates by role, but almost nothing checks that a specific mentor owns the specific session, assignment, join-request, or mock-interview being modified — a peer-mentor privilege-escalation pattern that repeats across five or more controllers.
- **File persistence will not survive production.** The combination of serverless hosting and local-disk Multer storage is fundamentally incompatible; this needs S3, Cloudinary, or an equivalent before any upload feature — avatars, resumes, submissions, resources — can be trusted in production.
- **No cascade-delete or referential-integrity strategy exists.** Every relationship in the schema (Course → Batch → Enrollment → Session → Assignment → Attendance → Certificate) is a bare `ObjectId` reference with no cleanup on delete.
- **The frontend has a genuine XSS surface.** The application already has a working `escHtml()` helper and uses it correctly in most places, but roughly seven render functions — announcements, notifications, session and course cards, join requests — bypass it. A single compromised admin account, or a mentor with announcement-adjacent permissions, could persist a payload visible to every user.
- **The demo-mode fallback blurs real and fabricated data.** This is intentional for offline demonstrations, but its heuristic auto-activation on a perceived backend outage (`frontend:1311`) risks a genuine production outage being silently masked as "working" during a live incident.

07 Performance Analysis

- **Unbounded, unpaginated list endpoints:** resources, alumni, office-hours, and several others carry no page-size cap, so a request such as `?limit=999999` can pull an entire collection.
- **An N+1 query pattern** appears in the leaderboard (`UserController.js:229-250`), and an in-memory aggregation anti-pattern appears in placement analytics (`placementController.js:122-171`), where full collections are loaded into Node and reduced in JavaScript rather than aggregated in MongoDB.
- **The 4,462-line single-file frontend** has no code-splitting or build step — every visitor downloads the full admin, mentor, and student application regardless of role, with no lazy-loading of rarely used pages such as the Resume Builder stub, Alumni, or Office Hours.
- **Duplicate Mongoose indexes** — on `User.status`, `User.studentProfile.enrollmentId`, and `Course.slug` — are each declared twice, wasting index-build time and disk space. This carries no functional bug but is worth a ten-minute cleanup.
- **No caching layer** was observed for course-catalog or analytics endpoints, both of which are read far more often than they are written.

08 Test Coverage Report



The suite that exists is well written — auth register, login, and me; course create, list, and module-add, including a 403 role check — it is simply narrow. Zero test coverage exists for batches, sessions, assignments (including submission and grading), attendance, placements, announcements, notifications, leads, reviews, resources, payments, certificates, join-requests, user management, analytics, companies, mock interviews, alumni, office hours, and search. Given that the ownership and validation gaps documented in Section 4 live almost entirely inside these untested controllers, test coverage and the bug list are directly correlated — closing this gap is the single highest-leverage engineering investment across Sprint 1 and Sprint 2.

09 Feature Priority Matrix

PRIORITY	CRITERIA	EXAMPLES FROM THIS AUDIT
CRITICAL	Security or financial exposure, live in production today	Password-reset token leak (B-01), unauthenticated lead mass-assignment (B-02), self-reported payments (B-03), cross-student data leak (B-04), upload persistence (B-05)
HIGH	Privilege escalation within a role, stored XSS, missing authorization on sensitive reads	Peer-mentor ownership gaps (B-06 through B-09), unscoped payment, certificate, and user reads (B-10 through B-12), unescaped innerHTML (B-14 through B-16), permissive CORS (B-17)
MEDIUM	Data integrity, missing validation on non-financial writes, N+1 or scale risk	Bulk validator rollout, cascade-delete strategy, batch-capacity race condition, leaderboard N+1
LOW	Polish, accessibility, dead code, logging hygiene	Keyboard and ARIA gaps, duplicate indexes, leftover console logging, dead code paths

10 Sprint Development Backlog

SPRINT 1 Stop the bleeding — security and data-integrity fixes · target two weeks

1. Fix B-01: never return the reset token in the API response; require SMTP to be configured before forgot-password ships to production, or gate token-in-body behind a server-side check that actually blocks it in production configuration.
2. Fix B-02: whitelist the fields `Lead.create` accepts from public input — first name, last name, email, phone, and message only.
3. Fix B-03: block direct status and amount edits on `updatePayment`; require a real gateway integration, or, if payments remain manual for launch, restrict edits to admin-only, reason-logged corrections with an audit trail.
4. Fix B-04: remove the unconditional `req.query.student` override in `mockInterviewController`; re-derive scope from `req.user` only.
5. Decide and implement real file storage — S3, Cloudinary, or equivalent — before any upload feature is considered complete; this replaces B-05.
6. Add the ownership check — `resource.mentor === req.user._id` unless admin — to sessions, assignments, join-requests, and mock interviews (B-06 through B-09).
7. Add route-level `authorize()` guards to the unscoped reads identified as B-10 through B-12.
8. Fix the seven unescaped-`innerHTML` sites using `escHtml()` (B-14 through B-16) — the highest-value, lowest-effort fix in this entire report.
9. Lock CORS: reject the `'*'` plus `credentials: true` combination in production configuration; require an explicit `CLIENT_URL`.
10. Attach express-validator rules to every payment route (B-18) and to every batch, session, certificate, placement, and assignment create/update route identified in the roughly thirty-route validator gap.

SPRINT 2 Harden and extend — reliability, test coverage, launch-blockers · target two weeks

1. Write Jest coverage for the twenty currently untested controllers, prioritizing those touched during Sprint 1 — sessions, assignments, payments, join-requests, mock interviews — so the fixes above are regression-protected.
2. Add cascade-delete, or soft-delete with orphan guards, across Course → Batch → Enrollment → Session → Assignment → Certificate → Attendance.
3. Fix the batch-enrollment race condition with an atomic `findOneAndUpdate` capacity check.
4. Surface notification and cron-job failures instead of silently swallowing them, across `notificationService.js` and the three affected jobs in `cronService.js` — at minimum, alert an admin dashboard widget on repeated failures.
5. Replace the unmaintained `xss-clean` package and upgrade Multer to the 2.x line; run `npm audit` and clear the remainder.

6. Build a real trainer-facing Reports page, at parity with the admin Reports page but scoped to each mentor's own batches.
7. Decide the fate of the Resume Builder and AI Assistant navigation entries — build a genuine version one, or remove the stub buttons. A "coming soon" toast reads as unfinished on a launch build rather than as a roadmap teaser.
8. Add pagination to every currently unbounded list endpoint — resources, alumni, office-hours, and the leaderboard.
9. Add keyboard and ARIA support to the primary click targets flagged in Section 4 — sidebar navigation, quick links, course rows — as a shared "clickable row" component; this is the largest single accessibility gap and is inexpensive to fix once.
10. Remove or gate the demo-mode auto-activation heuristic (frontend:1311) behind an explicit environment flag rather than an inferred outage, reducing the risk that a real production outage is silently hidden.

11 Development Roadmap

PHASE	FOCUS	EXIT CRITERIA
Sprint 1 Weeks 1–2	Security and financial-integrity fixes	All five Critical and seventeen High items closed; CORS locked down; validators attached to every money- and ownership-sensitive route
Sprint 2 Weeks 3–4	Reliability, test coverage, cascade integrity, accessibility baseline	Test coverage on previously untested controllers reaches 60% or higher; cascade-delete strategy is live; an accessibility pass covers the primary navigation
Sprint 3 Weeks 5–6	Feature completion for launch claims — real file storage, a payment gateway or a documented manual-only scope, SMTP wired up, a trainer Reports page	Every module listed in docs/FEATURES.md is either fully real or explicitly re-scoped or removed from the launch claim
Sprint 4+ Post-launch	New modules prioritized by business need — Resume Builder, a real Mock Test engine, an AI Assistant, Help Desk ticketing, an activity/audit log	Prioritized against actual user demand after launch, not pre-committed ahead of it

12 UI/UX Recommendations

- **Make role-hidden UI match role-enforced UI.** Every place a button is hidden client-side for a given role should have its server route locked down the same way — currently the reverse holds in several places (see B-10 through B-12).
- **Replace the remaining `prompt()` and `alert()` flows** — assignment submission and placement-drive application — with the application's existing `formModal()` and `toast()` pattern. These two flows are the only ones that fall back to native browser dialogs, and they read as unfinished next to the rest of the interface.
- **Standardize empty states.** Most lists correctly show a "nothing here yet" state; a few — join-request history, the calendar on a failed fetch — silently render blank instead, leaving it ambiguous whether data is missing or the page is broken.
- **Standardize error handling per page.** Several dashboard-load functions catch some of their parallel fetches into empty arrays while letting others throw to a page-level error handler. Choosing one behavior per page would keep a partial API outage from looking inconsistent to the user.
- **Make the demo dataset an explicit, clearly labeled mode** — a persistent banner, not merely a state triggered on detection — whenever it is active, so a demo session is never mistaken for live data by an administrator.
- **Resolve the Resume Builder and AI Assistant stub buttons before launch.** A "coming soon" toast is reasonable for a beta but reads as unfinished on a launch build; either ship a genuine first version or remove the navigation entry.